

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense

print("Imported Successfully")
```

Imported Successfully

```
import os
import cv2
import numpy as np

data = []
labels = []

categories = ["yes", "no"]

for category in categories:
    path = f"data/classification/{category}"

    label = categories.index(category)

    if not os.path.exists(path):
        print(f"Error: Path not found -> {path}")
        continue

    for img_name in os.listdir(path):

        img_path = os.path.join(path, img_name)

        img = cv2.imread(img_path)

        if img is None:
            continue

        img = cv2.resize(img, (128,128))

        data.append(img)
        labels.append(label)

X = np.array(data) / 255.0
y = np.array(labels)

print("Images shape:", X.shape)
print("Labels shape:", y.shape)
```

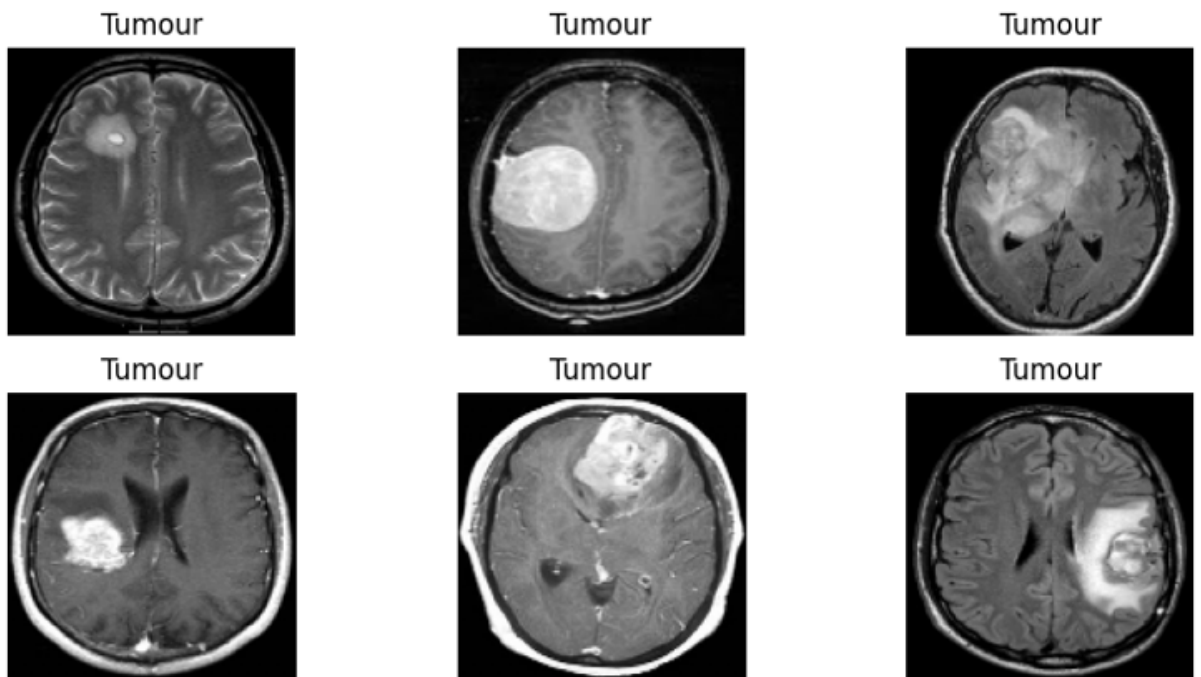
```
Images shape: (38, 128, 128, 3)
Labels shape: (38,)
```

```
plt.figure(figsize=(10,5))

for i in range(6):
    plt.subplot(2,3,i+1)

    plt.imshow(X[i])
    plt.title("Tumour" if y[i]==0 else "No Tumour")
    plt.axis("off")

plt.show()
```



```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128
model.add(MaxPooling2D())

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D())

model.add(Flatten())

model.add(Dense(64, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolution
super().__init__(activity_regularizer=activity_regularizer, **kwar
Model: "sequential"

```

Layer (type)	Output Shape	F
conv2d (Conv2D)	(None, 126, 126, 32)	
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	
conv2d_1 (Conv2D)	(None, 61, 61, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	
flatten (Flatten)	(None, 57600)	
dense (Dense)	(None, 64)	3,6
dense_1 (Dense)	(None, 1)	

Total params: 3,705,921 (14.14 MB)
Trainable params: 3,705,921 (14.14 MB)
Non-trainable params: 0 (0.00 B)

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=5,  
    validation_data=(X_test, y_test)  
)
```

Epoch 1/5

1/1  **3s** 3s/step - accuracy: 0.4667 - loss: 0.702

Epoch 2/5

1/1  **1s** 899ms/step - accuracy: 0.5333 - loss: 1.

Epoch 3/5

1/1  **1s** 1s/step - accuracy: 0.7333 - loss: 0.513

Epoch 4/5

1/1  **1s** 1s/step - accuracy: 0.7000 - loss: 0.571

Epoch 5/5

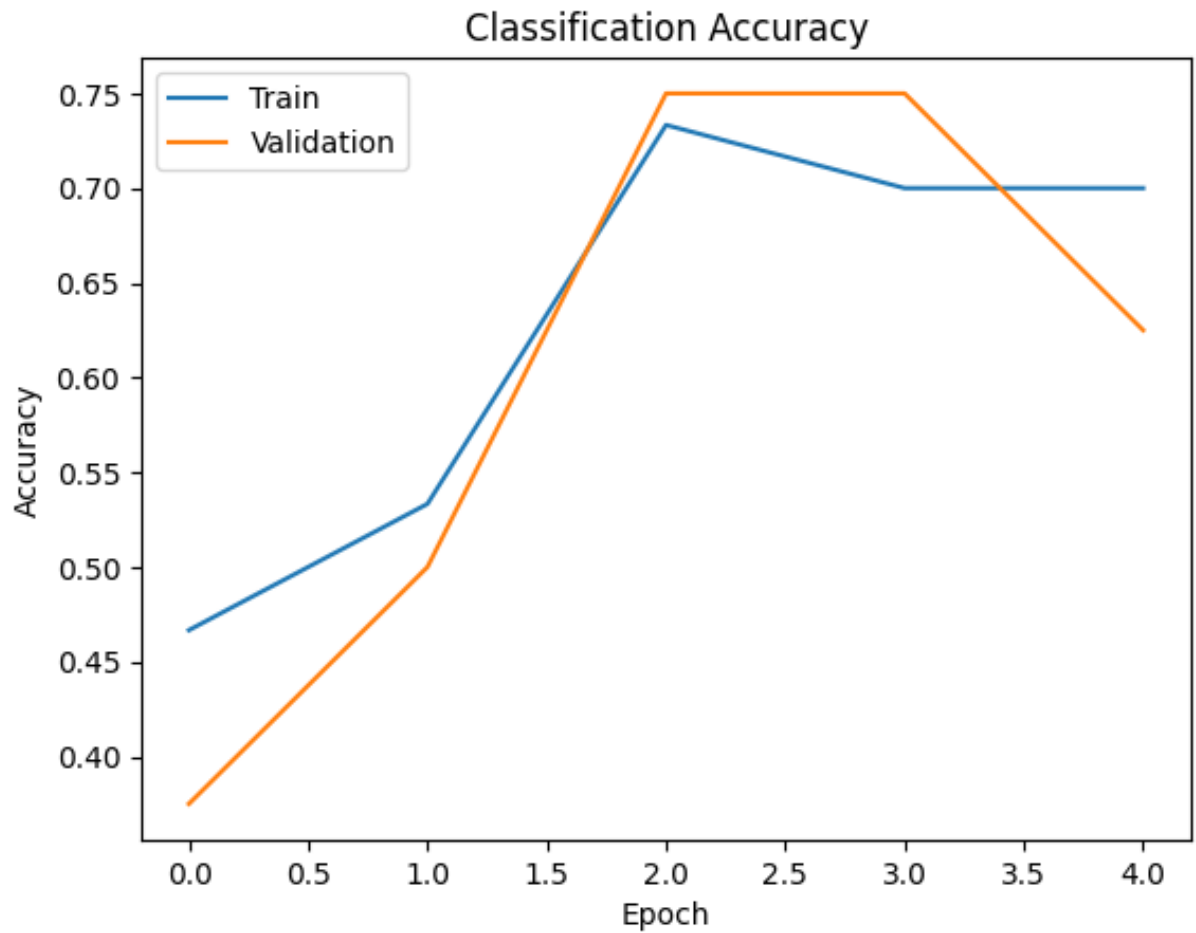
1/1  **1s** 785ms/step - accuracy: 0.7000 - loss: 0.

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title("Classification Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend(["Train", "Validation"])

plt.show()
```



```
sample = X_test[7]

prediction = model.predict(np.expand_dims(sample, axis=0))

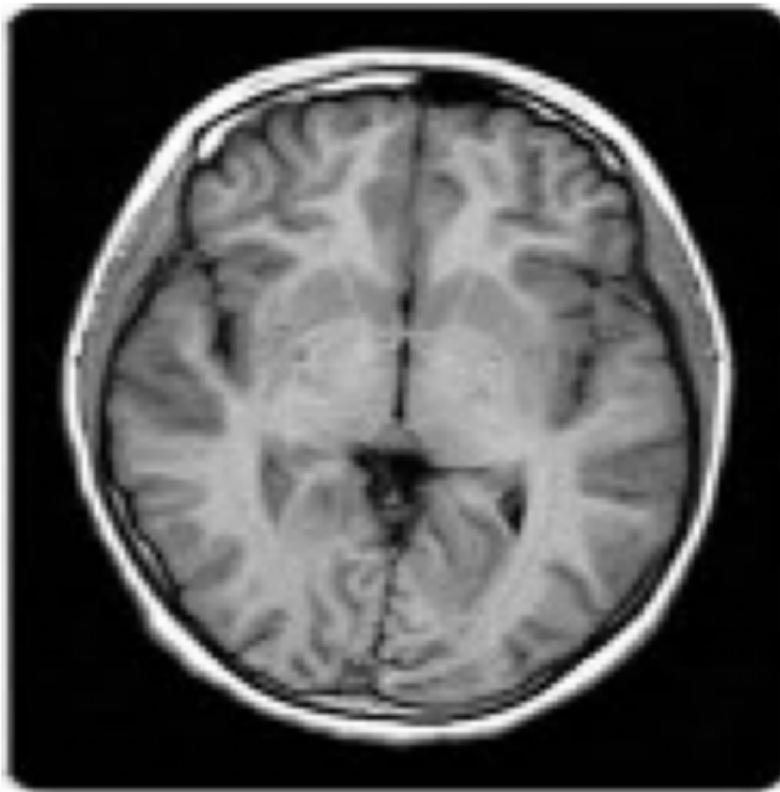
plt.imshow(sample)

if prediction > 0.5:
    plt.title("No Tumour")
else:
    plt.title("Tumour")

plt.axis("off")
plt.show()
```

1/1 ————— 0s 95ms/step

No Tumour



```
sample2 = X_test[2]

prediction = model.predict(np.expand_dims(sample2, axis=0))

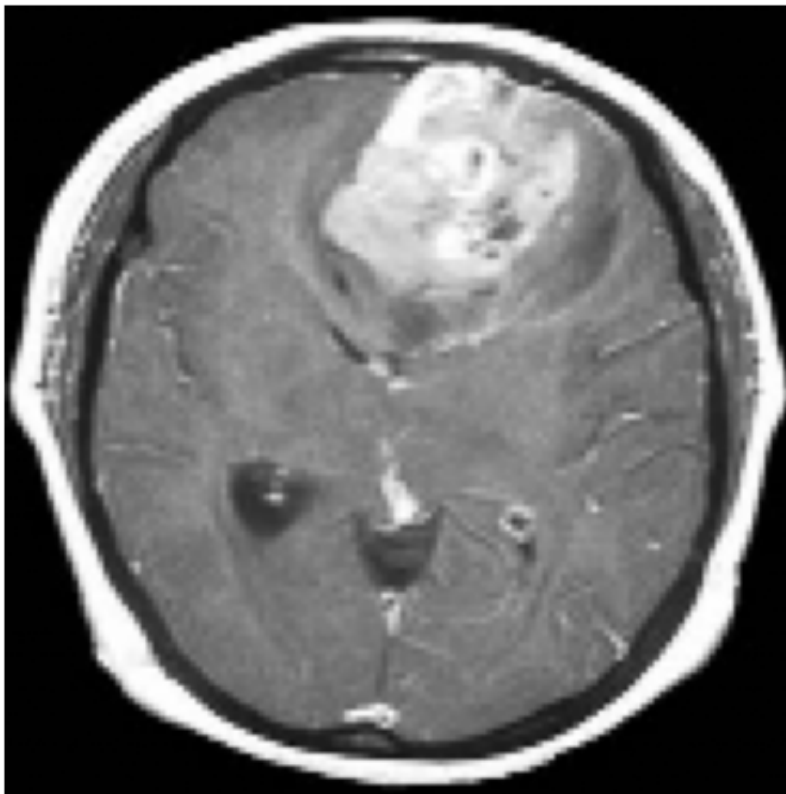
plt.imshow(sample2)

if prediction > 0.5:
    plt.title("No Tumour")
else:
    plt.title("Tumour")

plt.axis("off")
plt.show()
```

1/1 ————— 0s 44ms/step

Tumour



Start coding or [generate](#) with AI.

